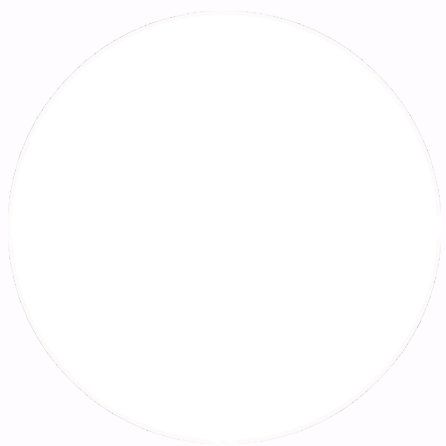


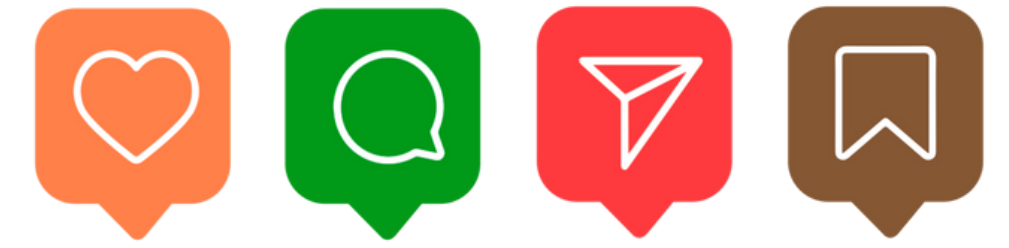
Grow in System Design

Learn 25+ concepts



@tauseeffayyaz 





Fundamentals of System Design

Latency vs Throughput, CAP Theorem, ACID Transactions, SQL vs NoSQL, Database Index, Strong vs Eventual Consistency, Idempotency, and Checksum.

Data Management & Scaling

Caching, Distributed Caching, Database Scaling, Data Replication, Data Redundancy, Database Sharding, and Consistent Hashing.

Networking & Communication

Content Delivery Network, Domain Name System, Proxy Server, Load Balancing, Rate Limiting, Message Queues, WebSockets, API Gateway, and REST vs RPC.

Reliability & Fault Tolerance

HeartBeat, Circuit Breaker, Distributed Locking, and Consensus Algorithms.

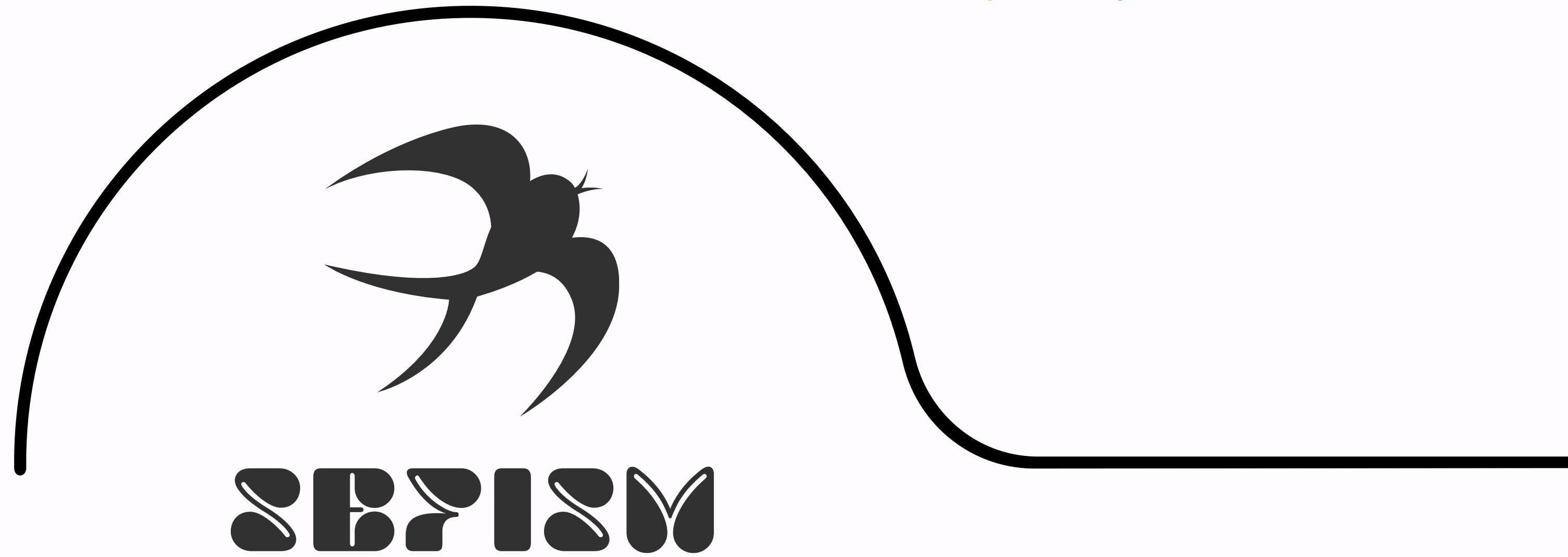
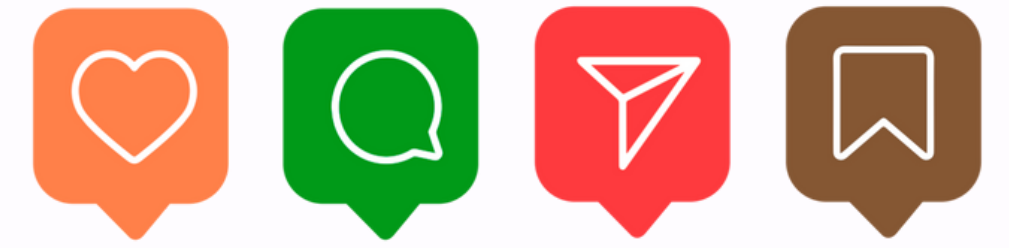
Architectural Patterns

Microservices Architecture and Batch vs Stream Processing.



@tauseeffayyaz 





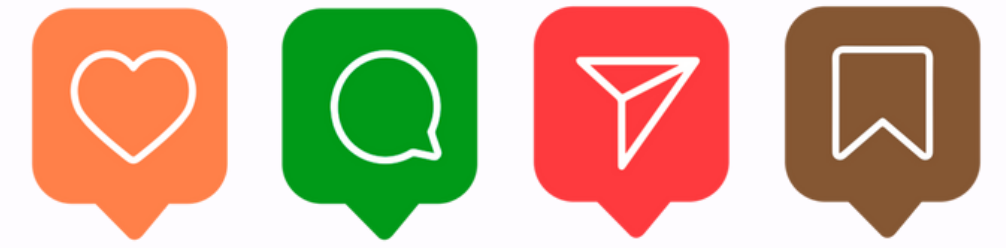
[read full article](#)

Latency vs Throughput

Latency measures delay, throughput measures capacity. Balancing both is crucial in performance-sensitive applications.

Optimize systems for speed and efficiency.





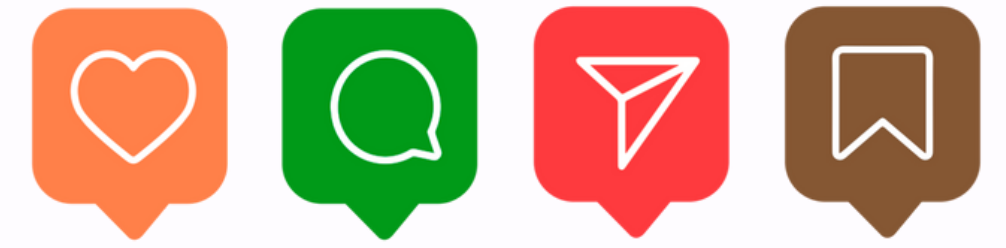
[read full article](#)

CAP Theorem

Explains trade-offs between Consistency, Availability, and Partition Tolerance in distributed systems.

Guide architecture choices in distributed databases.





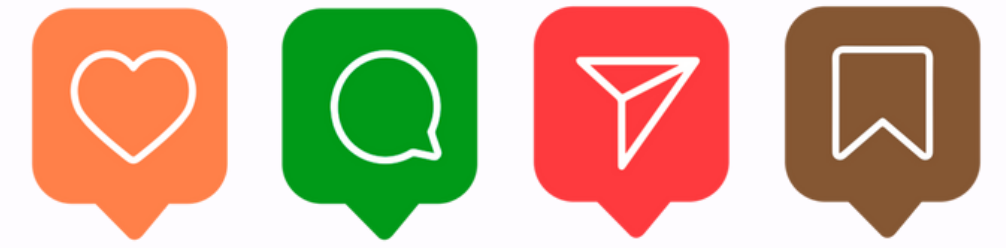
[read full article](#)

ACID Transactions

Guarantees reliable database operations: Atomicity, Consistency, Isolation, Durability.

Ensure correctness in financial transactions.





SEPRISM

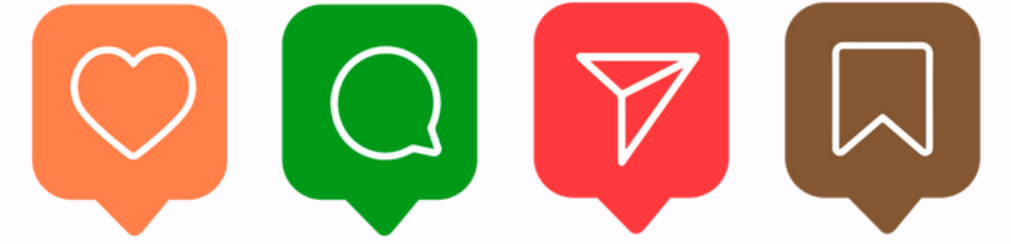
[read full article](#)

SQL vs NoSQL

SQL is structured and relational; NoSQL is flexible, schema-less, and scalable.

Choose storage type based on data needs.





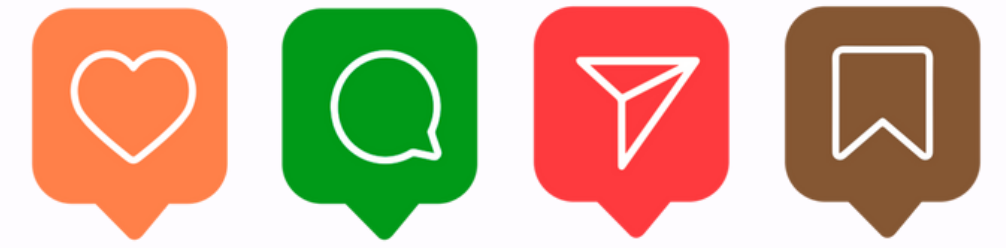
[read full article](#)

Database Index

Indexes improve query performance by quickly locating data without scanning entire tables.

Speed up frequent database queries.





SEPRISM

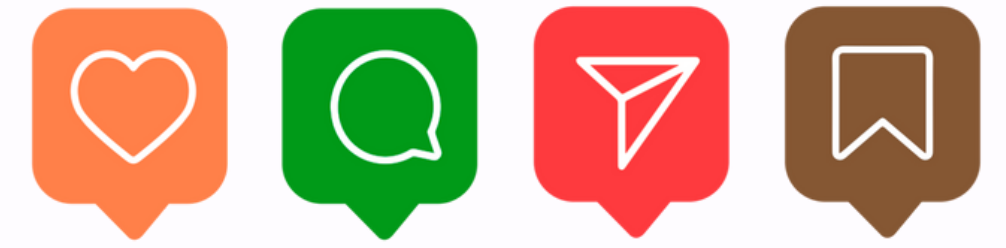
[read full article](#)

Strong vs Eventual Consistency

Strong consistency ensures accuracy, eventual consistency favors availability in distributed systems.

Select consistency based on system requirements.





SEPRISM

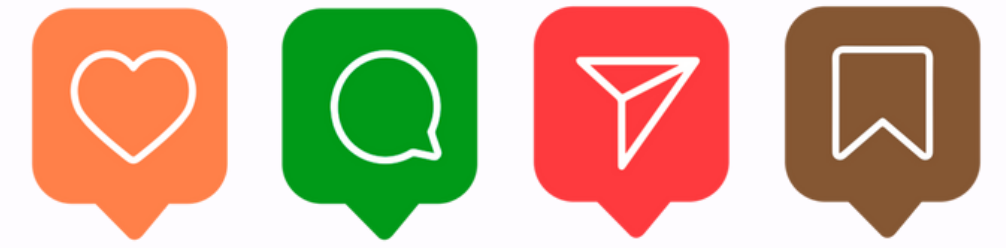
[read full article](#)

Idempotency

An idempotent operation can be repeated without changing the result.

Prevent duplicate effects in retries.





SEPRISM

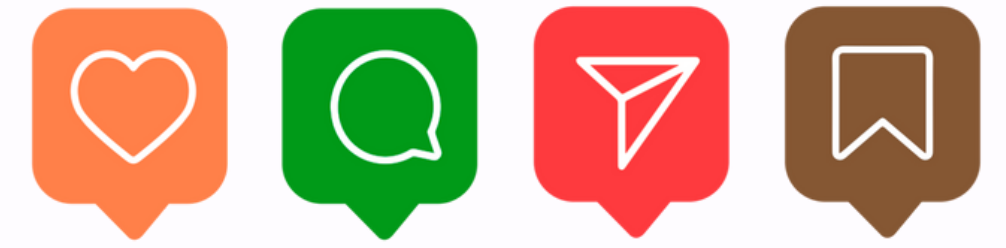
[read full article](#)

Checksum

A small value derived from data to detect corruption or errors during transfer/storage.

Validate integrity of transmitted data.





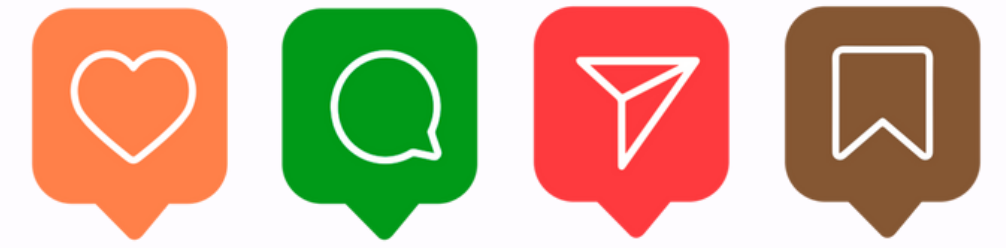
[read full article](#)

Caching (Basics)

Stores frequently accessed data in memory to reduce latency and server load.

Improve speed by reducing database hits.





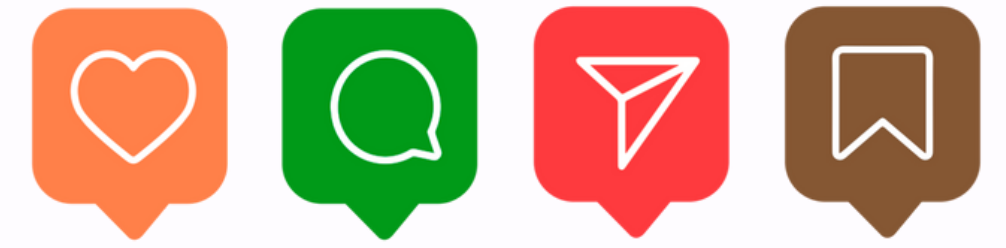
[read full article](#)

Distributed Caching

Caches spread across servers provide scalability and higher availability.

Scale cache for large user bases.





SEPRISM

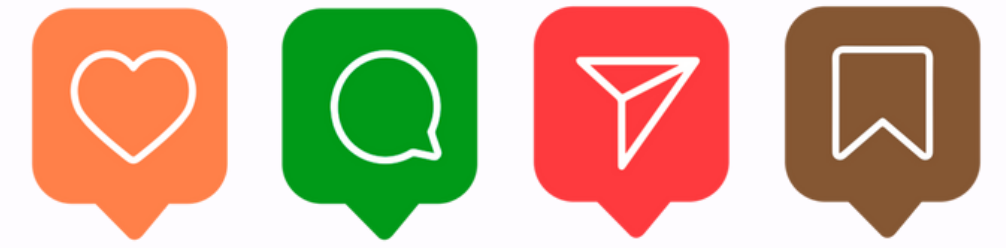
[read full article](#)

Database Scaling

Techniques to expand databases vertically or horizontally for larger workloads.

Handle growing data and traffic efficiently.





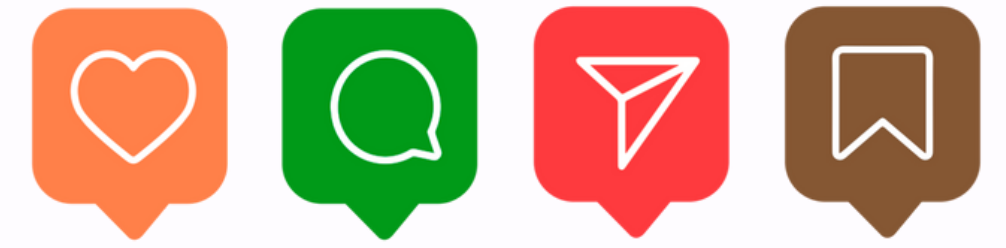
[read full article](#)

Data Replication

Copying data across servers improves availability and fault tolerance.

Maintain system uptime during server failures.





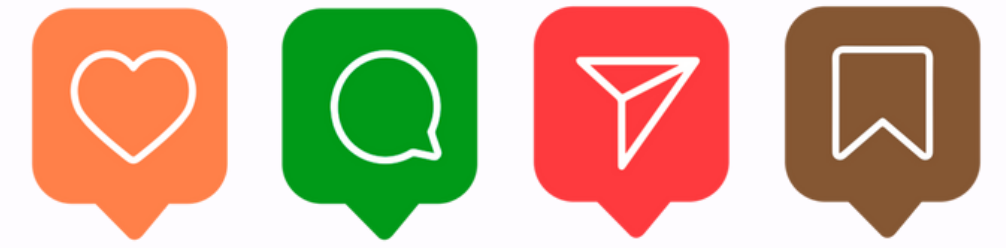
[read full article](#)

Data Redundancy

Storing duplicate data ensures backup and fault tolerance but adds cost.

Avoid data loss during outages.





SEPRISM

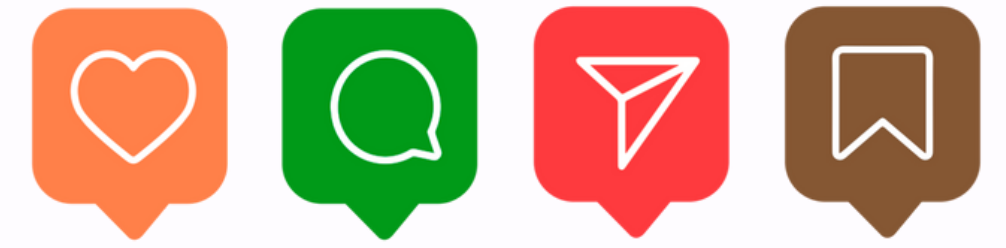
[read full article](#)

Database Sharding

Storing duplicate data ensures backup and fault tolerance but adds cost.

Avoid data loss during outages.





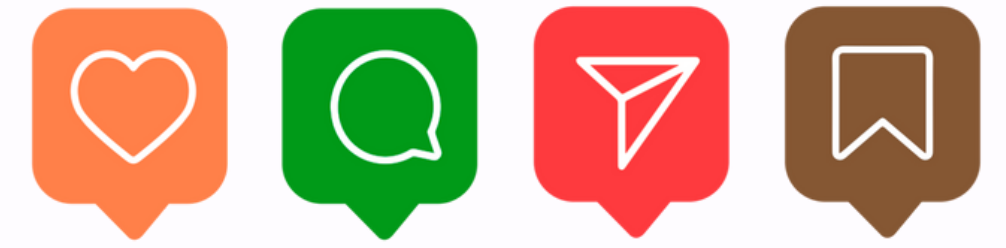
[read full article](#)

Consistent Hashing

Efficiently maps data across nodes, minimizing re-distribution during scaling.

Balance load across distributed systems.





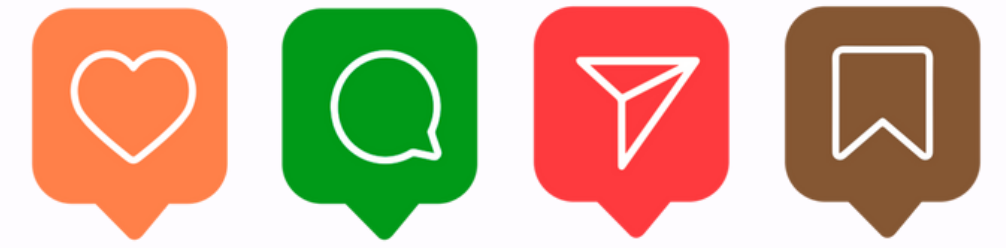
[read full article](#)

Content Delivery Network (CDN)

Delivers cached static and dynamic content closer to users worldwide.

Reduce latency for global content delivery.





SEPRISM

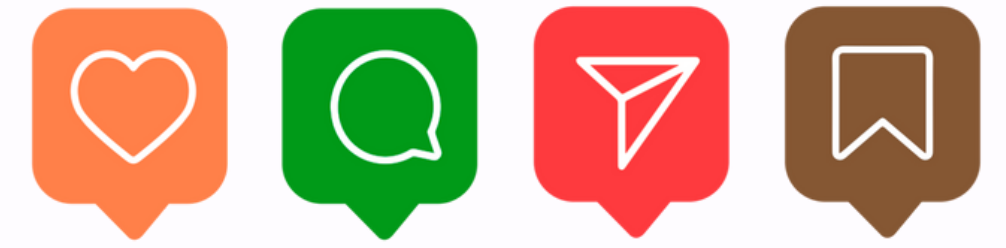
[read full article](#)

Domain Name System (DNS)

Translates human-readable names into machine-friendly IP addresses.

Route traffic reliably across the internet.





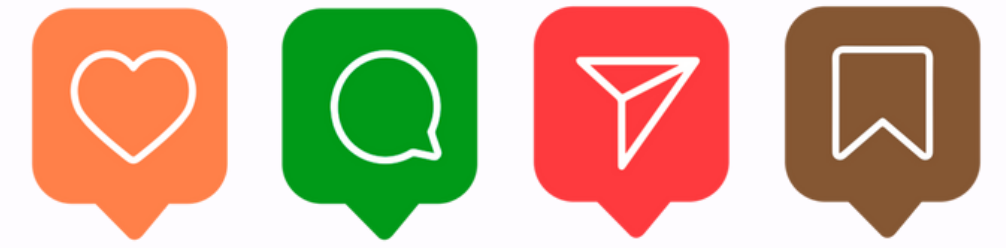
[read full article](#)

Proxy Server

Acts as an intermediary for client requests to servers.

Improve security, caching, and load handling.





SEPRISM

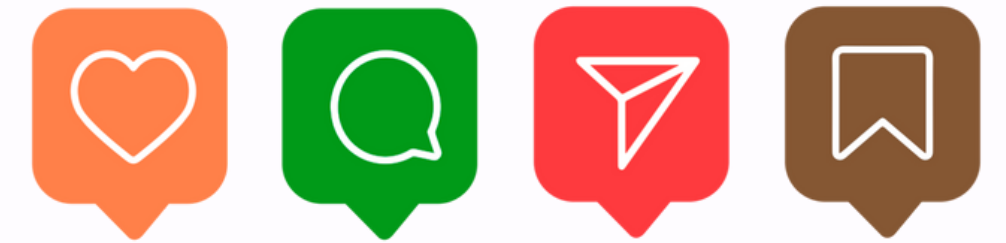
[read full article](#)

Load Balancing

Distributes client requests evenly across multiple servers.

Prevent overload on a single server.





SEPRISM

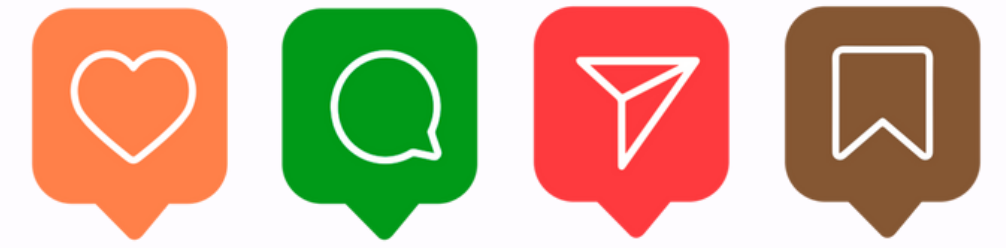
[read full article](#)

Rate Limiting

Controls how many requests a client can make.

Protect APIs from abuse or misuse.





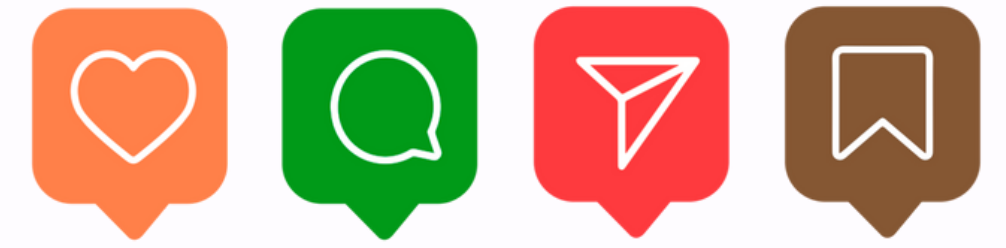
[read full article](#)

Message Queues

Enable asynchronous communication between services via queued tasks.

Decouple services for scalability and reliability.





SEPRISM

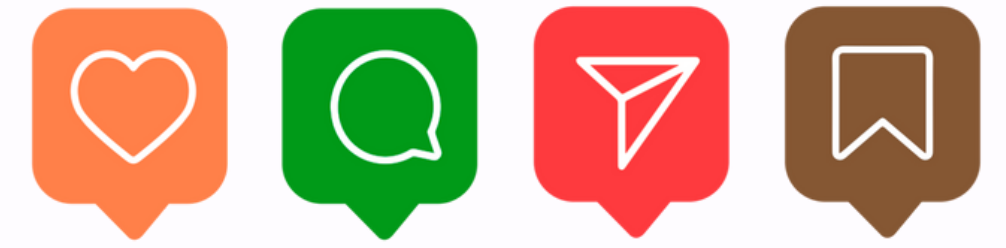
[read full article](#)

WebSockets

Provide persistent, bidirectional communication between client and server.

Enable real-time updates in applications.





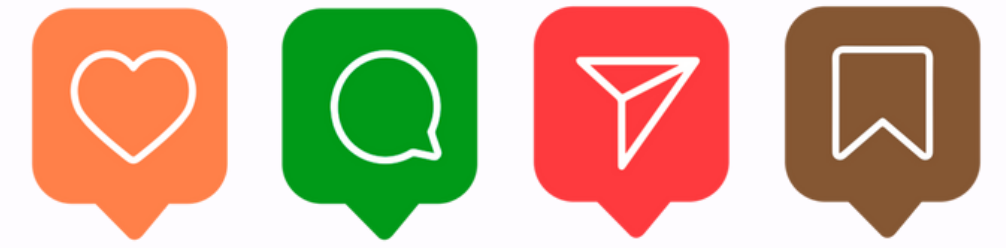
[read full article](#)

API Gateway

Centralized entry point for managing API requests and policies.

Secure and route microservice communication.





SEPRISM

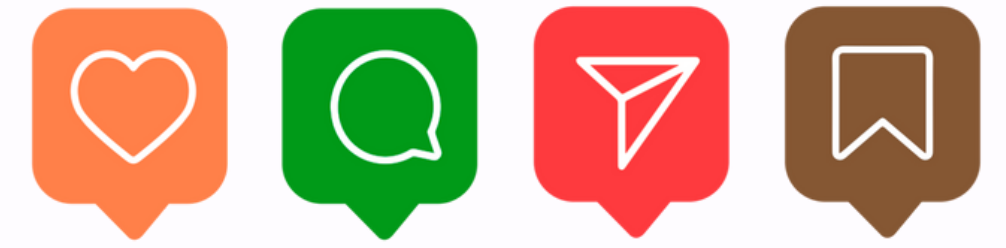
[read full article](#)

REST vs RPC

REST focuses on resources, RPC on actions/procedures.

Choose protocol style for service interaction.





SEPRISM

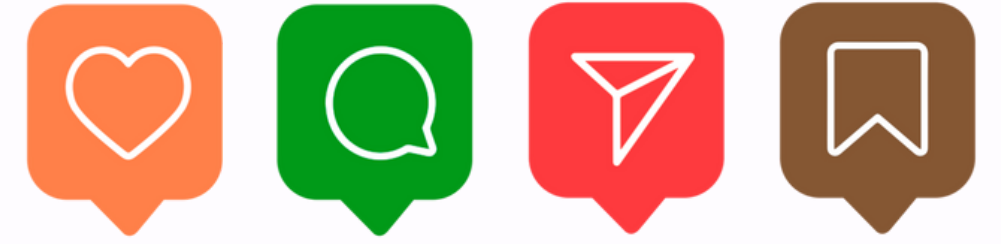
[read full article](#)

HeartBeat

Periodic signals confirm whether a system component is alive.

Detect failures in distributed environments.





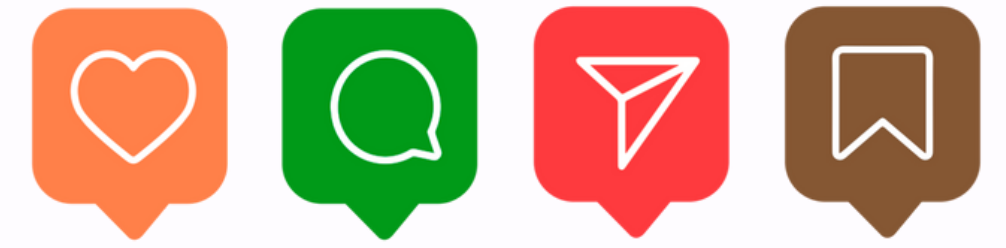
[read full article](#)

Circuit Breaker

Prevents cascading failures by halting calls to unhealthy services.

Protect systems from repeated failures.





SEPRISM

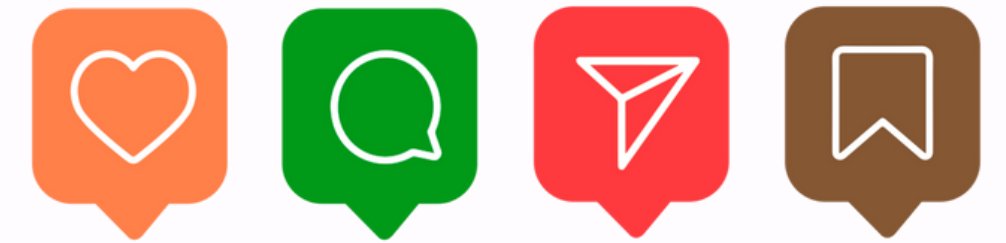
[read full article](#)

Distributed Locking

Ensures only one process accesses a shared resource at a time.

Avoid race conditions in distributed apps.





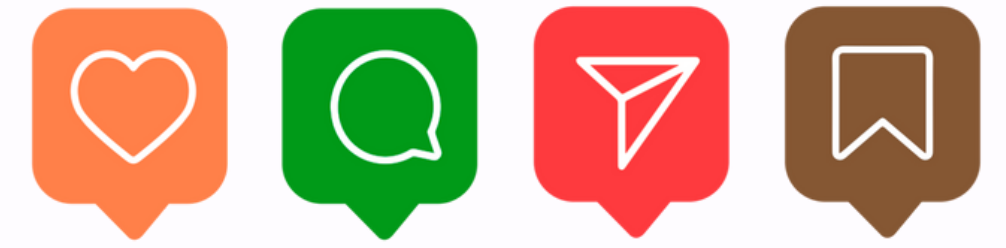
[read full article](#)

Consensus Algorithms

Help distributed nodes agree on a single truth.

Maintain consistency across replicated services.





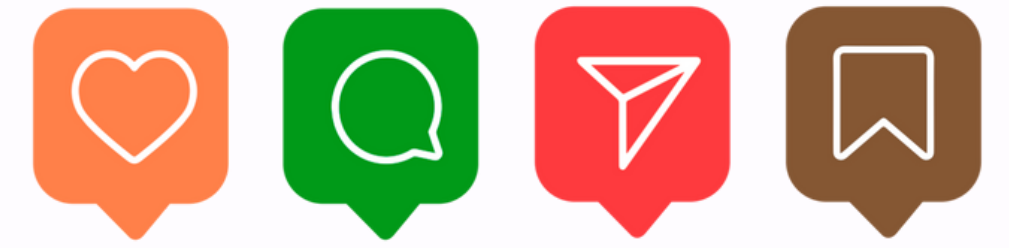
[read full article](#)

Microservices Architecture

Breaks applications into small, independent, deployable services.

Build modular, scalable, maintainable applications.





[read full article](#)

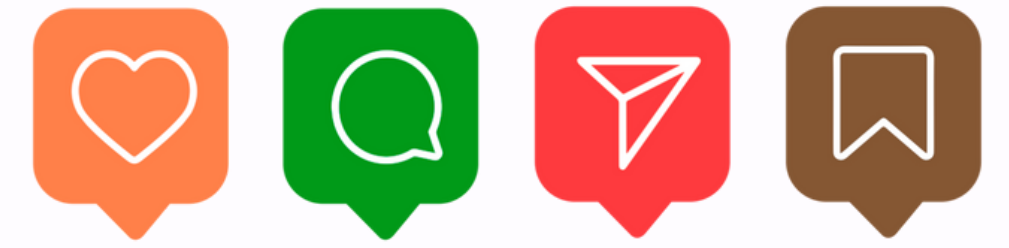
Batch vs Stream Processing

Batch handles data in bulk, stream in real-time.

Choose processing style based on workload.



@tauseeffayyaz



About Tauseef Fayyaz

Full-Stack Engineer, Ranked #1 in Computer Engineering (Pakistan), Career Mentor for Aspiring Engineers, Tech Content Creator

You can request a dedicated mentorship session with me for guidance, support, or just honest career advice.

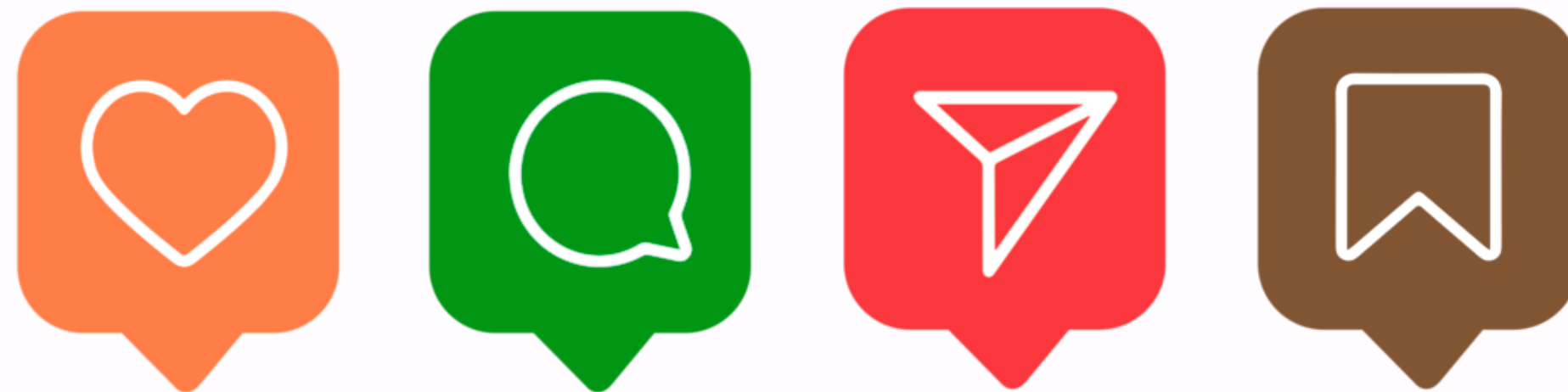
<https://topmate.io/tauseeffayyaz/>



@tauseeffayyaz



**Your support keeps
me motivated.**



<https://www.linkedin.com/in/tauseeffayyaz/>